

AWS Migration and Hybrid Architecture

MODULE 8

BCSE355L – AWS SOLUTION ARCHITECT

AWS Migration

AWS MIGRATION

- A migration strategy is the approach used to migrate a workload into the AWS Cloud. There are seven migration strategies for moving applications to the cloud, known as the 7 Rs:
 - Retire
 - Retain
 - Rehost
 - Relocate
 - Repurchase
 - Replatform
 - Refactor or re-architect

AWS MIGRATION

- Common strategies for large migrations include rehost, replatform, relocate, and retire.
- Refactor is not recommended for large migrations because it involves modernizing the application during the migration.
- This is the most complex of the migration strategies, and it can be complicated to manage for a large number of applications.
- Instead, we recommend rehosting, relocating, or replatforming the application and then modernizing the application after the migration is complete.
- Selecting migration strategies is critical to a large migration.
- Selected migration strategies in the mobilize phase or during the initial portfolio assessment.
- This section reviews each migration strategy and their common use cases.

AWS MIGRATION STRATEGIES

- **Retire**
- This is the migration strategy for the applications that you want to decommission or archive. Retiring the application means that you can shut down the servers within that application stack. The following are common use cases for the retire strategy:
 - There is no business value in retaining the application or moving it to cloud.
 - To eliminate the cost of maintaining and hosting the application.
 - To reduce the security risks of operating an application that uses an operating system (OS) version or components that are no longer supported.
 - To retire applications based on their performance. There has been no inbound connection to the application for the last 90 days.

AWS MIGRATION STRATEGIES

- **Retain**

- This is the migration strategy for applications that you want to keep in your source environment or applications that you are not ready to migrate. You might choose to migrate these applications in the future.
- The following are common use cases for the retain strategy:
 - **Security and compliance** – You might want to retain applications in order to remain in compliance with data residency requirements.
 - **High risk** – You might decide to retain an application because it requires a detailed assessment and plan prior to migration.
 - **Dependencies** – You might decide to retain an application if you need to migrate one or more other applications first.

AWS MIGRATION STRATEGIES

- **Retain**

- The following are common use cases for the retain strategy:

- **Applications that are recently upgraded** – You might want to postpone migrating the application until the next technical refresh because you recently invested in upgrading your current system.
- **No business value to migrate** – There is no business value for migrating some applications to the cloud, such as those with only a few internal users.
- **Plans to migrate to software as a service (SaaS)** – You might choose retain an application until the SaaS version is released by the vendor. This is a common strategy for vendor-based applications.
- **Unresolved physical dependencies** – You might choose to retain an application that is dependent on specialized hardware that does not have a cloud equivalent, such as machines in a manufacturing plant.

AWS MIGRATION STRATEGIES

- **Retain**

- The following are common use cases for the retain strategy:

- **Mainframe or mid-range applications and non-x86 Unix applications** – These applications require careful assessment and planning before migrating them to the cloud. Examples of mid-range applications include IBM AS/400 and Oracle Solaris.
- **Performance** – You might want to retain applications based on their performance. For example, you might want to keep zombie or idle applications in your source environment.

AWS MIGRATION STRATEGIES

- **Rehost**

- This strategy is also known as lift and shift. Using this strategy, you move your applications from your source environment to the AWS Cloud without making any changes to the application.
 - For example, you migrate your application stack from on-premises to the AWS Cloud.
- With rehost, you can migrate a large number of machines from multiple source platforms (physical, virtual, or another cloud) to the AWS Cloud without worrying about compatibility, performance disruption, long cutover windows, or long-distance data replications.
- Your application continues to serve users while the workloads are being migrated, which minimizes disruption and downtime. The downtime depends on your cutover strategy.

AWS MIGRATION STRATEGIES

- **Rehost**

- This strategy helps you to scale your applications without implementing any cloud optimizations that could save you time or money. Applications are easier to optimize or re-architect when they are already running in cloud because it is easier to integrate to AWS services and manage your workloads.
- Automate rehosting by using the following services:
 - **AWS Application Migration Service**
 - **AWS Cloud Migration Factory Solution**
 - **VM Import/Export**

AWS MIGRATION STRATEGIES

- **Relocate**

- Using this strategy, you can transfer a large number of servers, comprising one or more applications, at a given time from on-premises platform to a cloud version of the platform.
- You can also use the relocate strategy to move instances or objects to a different virtual private cloud (VPC), AWS Region, or AWS account.
- For example, you can use this strategy to transfer an Amazon Relational Database Service (Amazon RDS) DB instance to another VPC or AWS account.

AWS MIGRATION STRATEGIES

- **Relocate**

- The relocate strategy doesn't require that you purchase new hardware, rewrite applications, or modify your existing operation.
- During relocation, the application continues to serve users, which minimizes disruption and downtime.
- Relocate is the quickest way to migrate and operate your workload in the cloud because it does not impact the overall architecture of your application.

AWS MIGRATION STRATEGIES

- **Repurchase**

- This strategy is also known as drop and shop. You replace your application with a different version or product.
- The new application should provide more business value than the existing, on-premises application, including features such as accessibility from anywhere, no infrastructure to maintain, and pay-as-you-go pricing models.
- Repurchasing the application typically reduces costs associated with maintenance, infrastructure, and licensing.

AWS MIGRATION STRATEGIES

- **Repurchase**

- The following are common use cases for the repurchase migration strategy:
 - **Moving from a traditional license to SaaS** – This removes the burden of managing and maintaining the infrastructure and helps reduce licensing issues.
 - **Version upgrades or third-party equivalents** – By replacing your existing on-premises application with the vendor's latest version or third-party equivalent in the cloud, you can leverage new features, integrate with cloud services, and scale the application more easily.
 - **Replacing a custom application** – You can avoid recoding and re-architecting a custom application by repurchasing a vendor-based SaaS or cloud-based application.

AWS MIGRATION STRATEGIES

- **Repurchase**
- Before purchasing, you need to assess the application according to your business requirements, especially security and compliance.
- After you purchase the new application, the following are the next steps:
 - Training your team and users with the new system
 - Migrating your data to the newly purchased application
 - Integrating the application into your authentication services, such as Microsoft Active Directory, to centralize authentication
 - Configuring networking to help secure communication between the purchased application, your users, and your infrastructure
- Typically, the application vendor helps you with these activities for a smooth transition.

AWS MIGRATION STRATEGIES

- **Replatform**

- This strategy is also known as lift, tinker, and shift or lift and reshape.
- Using this migration strategy, you move the application to the cloud, and you introduce some level of optimization in order to operate the application efficiently, to reduce costs, or to take advantage of cloud capabilities.
- For example, you might replatform a Microsoft SQL Server database to Amazon RDS for SQL Server.
- Using this strategy, you might make a few or many changes to the application, depending on your business goals and your target platform.

AWS MIGRATION STRATEGIES

- **Replatform**

- The following are common use cases for the Replatform migration strategy:
 - You want to save time and reduce cost by moving to a fully managed service or serverless service in the AWS Cloud.
 - You want to improve your security and compliance stance by upgrading your operating system to the latest version. By using End-of-Support Migration Program (EMP) for Windows Server, you can migrate your legacy Windows Server applications to the latest, supported versions of Windows Server on AWS, without any code changes. You can use this decision tree in the AWS EMP for Windows Server User Guide to help you determine your EMP workloads.
 - You can reduce costs by using AWS Graviton Processors custom-built processors developed by AWS.

AWS MIGRATION STRATEGIES

- **Replatform**

- The following are common use cases for the Replatform migration strategy:
 - You can reduce costs by moving from a Microsoft Windows operating system to a Linux operating system. You can port your .NET Framework applications to .NET Core, which can run on a Linux operating system. Porting Assistant for .NET is an analysis tool that helps you port your applications to Linux.
 - You can improve performance by migrating virtual machines into containers, without making any code changes. You can modernize your .NET and Java applications into containerized applications by using the AWS App2Container migration tool
- The Replatform strategy keeps your legacy application running without compromising security and compliance.
- Replatform reduces cost and improves performance by migrating to a managed or serverless service, moving virtual machines to container, and avoiding licensing expenses.

AWS MIGRATION STRATEGIES

- **Refactor or re-architect**

- Using this strategy, you move an application to the cloud and modify its architecture by taking full advantage of cloud-native features to improve agility, performance, and scalability. This is driven by strong business demand to scale, accelerate product and feature releases, and to reduce costs.
- The following are common use cases for the refactor migration strategy:
 - The legacy mainframe application can no longer address the demand of the business due to its limitations or is expensive to maintain.
 - You have a monolith application that is already hindering efforts to deliver product quickly or address customer needs and demands.

AWS MIGRATION STRATEGIES

- **Refactor or re-architect**

- The following are common use cases for the refactor migration strategy:
 - You have a legacy application that nobody knows how to maintain, or the source code is unavailable.
 - The application is difficult to test, or test coverage is very low. This affects the quality and delivery of new application features and fixes. By redesigning the application for the cloud, you can increase the test coverage and integrate automated testing tools.
 - For security and compliance reasons, when moving a database to the cloud, you might need to extract some tables (such as customer information, patient, or patient diagnosis tables) and retain those tables on premises. In this situation, you need to refactor your database in order to separate the tables that will be migrated from those that will be retained on premises.

AWS Hybrid

AWS Hybrid Architecture

- AWS hybrid cloud services deliver a consistent AWS experience wherever you need it – from the cloud, to on premises, and at the edge.
- Select from the broadest set of compute, networking, storage, security, identity, data integration, management, monitoring, and operations services to build hybrid architectures that meet your specific requirements and use cases.
- Can also use **AWS Solutions for Hybrid and Multicloud** to simplify and centralize the management of resources across hybrid and edge environments with resources on other clouds.

AWS Hybrid Architecture - Benefits

- **Accelerate digital transformation** - Bring cloud infrastructure and services where you need them to help your digital transformation projects get up and running faster. Modernize and manage all your applications with AWS, even those running in specific locations due to data residency, local processing, and latency requirements.
- **Improve IT and developer productivity** - Watch productivity go up and costs go down. Provide developers with a common platform for building, deploying, and managing applications and give IT the same services, and tools for operating infrastructure on AWS, on premises, at the edge, and on other clouds.

AWS Hybrid Architecture - Benefits

- **Deliver differentiated services and experiences** - Enable interactive and responsive applications and deliver new, unique offerings to users faster. Deploy your applications, infrastructure, and services on premises, near large metro centers, and at the edge of 5G networks so they are closer to your end-users.

AWS Hybrid cloud services - Compute

- **AWS Outposts**

- AWS Outposts allows you to run AWS infrastructure and services on premises for a consistent hybrid experience.

- **AWS Wavelength**

- AWS Wavelength embeds AWS compute and storage services at the edge of 5G networks for a faster application response time.

- **AWS Local Zones**

- AWS Local Zones put AWS services near large population and industry centers so you can reach local users in milliseconds.

- **AWS Snow**

- AWS Snow devices collect and process data between the edge and AWS so that your apps run in all conditions.

AWS Hybrid cloud services - Container

- **Amazon ECS Anywhere**

- Amazon Elastic Container Service (ECS) Anywhere provides consistent tooling and APIs for container-based applications - on premises and in the cloud.

- **Amazon EKS Anywhere**

- Amazon Elastic Kubernetes Service (EKS) Anywhere enables you to easily create and operate Kubernetes clusters on- premises, reducing support costs and eliminating redundant effort.

AWS Hybrid cloud services - Storage

- **AWS Storage Gateway**

- AWS Storage Gateway simplifies hybrid cloud storage management and reduces costs by enabling on premises workloads to use AWS storage.

- **AWS Backup**

- AWS Backup makes it easy to centralize and automate the back up of data across AWS in the cloud and on premises.

- **AWS DataSync**

- AWS DataSync accelerates and automates data transfer from your on premises systems into or out of AWS data storage for processing.

- **AWS Transfer Family**

- AWS Transfer Family enables file transfers into and out of Amazon Simple Storage Service (S3) or Amazon Elastic File System (EFS) over SFTP, FTPS, and FTP protocols.

AWS Hybrid cloud services - Networking

- **AWS Direct Connect**

- AWS Direct Connect establishes a private virtual interface from your on premises network to your Amazon Virtual Private Cloud (VPC).

- **Amazon Route 53 Resolver**

- Amazon Route 53 Resolver delivers recursive DNS to your Amazon VPC and on premises networks.

AWS Hybrid cloud services - Management

- **AWS Directory Service** - AWS Directory Service enables you to connect AWS resources with on premises Microsoft Active Directory and manage policies with existing tools.
- **Amazon IAM** - AWS Identity and Access Management (IAM) lets you use existing identity systems to securely manage access to AWS services
- **AWS Systems Manager** - AWS Systems Manager is a centralized hub for tracking and resolving operational issues across your AWS and on- premises resources.
- **AWS OpsWorks** - AWS OpsWorks uses Chef and Puppet to automate application configuration and operation, both on premises and on AWS.
- **AWS Code Deploy** - AWS CodeDeploy automates code deployments to Amazon EC2 and on premises instances to help you maintain application uptime.
- **Amazon CloudWatch** - Amazon CloudWatch is an observability service that unifies your view of resources, applications, and services running on AWS and on- premises.
- **AWS X-Ray** - AWS X-Ray runs on premises or in a Docker container to relay trace data from your instrumented application for debugging.

AWS Hybrid cloud services

○ Use Cases

- Support low-latency applications
- Process data locally
- Meet data residency requirements
- Extend your data center
- Move enterprise applications to the cloud

AWS
DMS

AWS DATABASE MIGRATION SERVICE

- AWS Database Migration Service (AWS DMS) is a cloud service that makes it possible to migrate relational databases, data warehouses, NoSQL databases, and other types of data stores. You can use AWS DMS to migrate your data into the AWS Cloud or between combinations of cloud and on-premises setups.
- With AWS DMS, you can discover your source data stores, convert your source schemas, and migrate your data.
 - To discover your source data infrastructure, you can use DMS Fleet Advisor. This service collects data from your on-premises database and analytic servers, and builds an inventory of servers, databases, and schemas that you can migrate to the AWS Cloud.

AWS DATABASE MIGRATION SERVICE

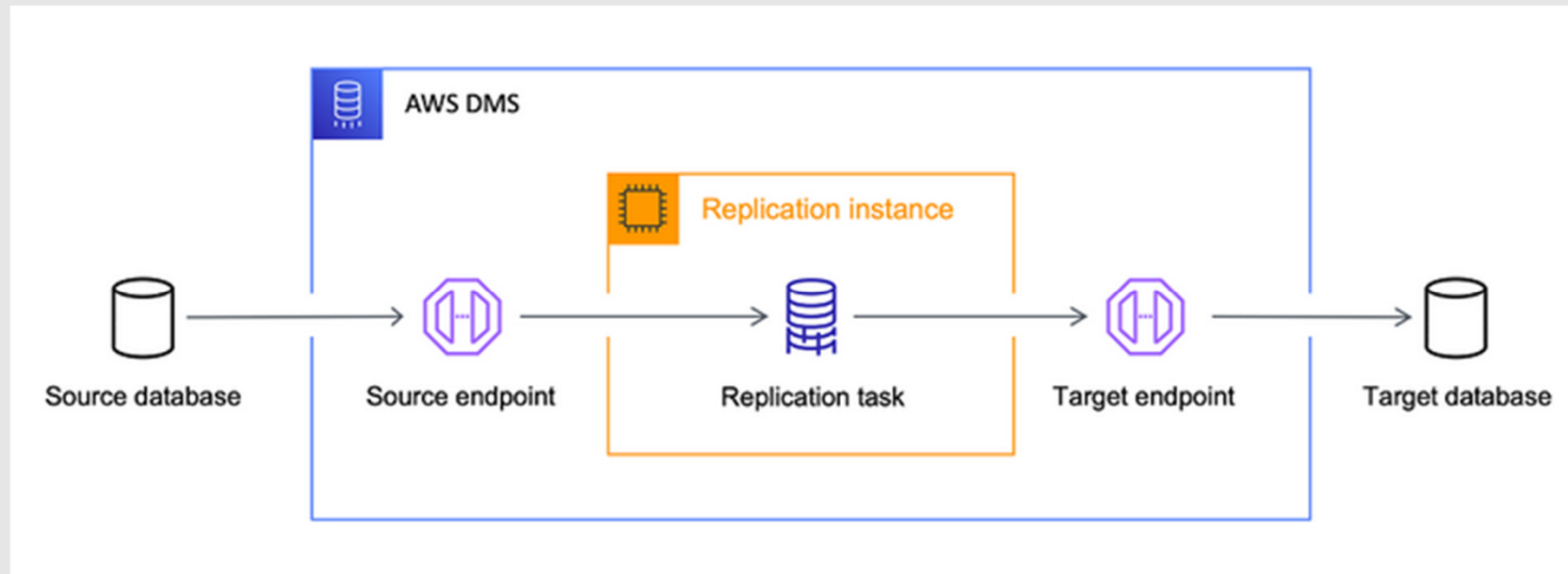
- To migrate to a different database engine, you can use DMS Schema Conversion. This service automatically assesses and converts your source schemas to a new target engine. Alternatively, you can download the AWS Schema Conversion Tool (AWS SCT) to your local PC to convert your source schemas.
- After you convert your source schemas and apply the converted code to your target database, you can use AWS DMS to migrate your data. You can perform one-time migrations or replicate ongoing changes to keep sources and targets in sync. Because AWS DMS is a part of the AWS Cloud, you get the cost efficiency, speed to market, security, and flexibility that AWS services offer.

AWS DATABASE MIGRATION SERVICE

- At a basic level, AWS DMS is a server in the AWS Cloud that runs replication software.
- You create a source and target connection to tell AWS DMS where to extract data from and where to load it.
- Next, you schedule a task that runs on this server to move your data.
- AWS DMS creates the tables and associated primary keys if they don't exist on the target.
- You can create the target tables yourself if you prefer. Or you can use AWS Schema Conversion Tool (AWS SCT) to create some or all of the target tables, indexes, views, triggers, and so on.

AWS DATABASE MIGRATION SERVICE

- The following diagram illustrates the AWS DMS replication process.



AWS DATABASE MIGRATION SERVICE

- The following diagram illustrates the AWS DMS replication process.
 - **AWS Regions that support AWS DMS** – For information about what AWS Regions support AWS DMS, see Working with an AWS DMS replication instance.
 - **Cost of database migration** – For information on the cost of database migration, see the AWS Database Migration Service pricing page
 - **AWS DMS features and benefits** – For information about AWS DMS features and benefits, see AWS Database Migration Service Features
 - **Available database options** – To learn more about the variety of database options available on Amazon Web Services

AWS DMS COMPONENTS

- **Database discovery**
- DMS Fleet Advisor collects data from multiple database environments to provide insight into your data infrastructure. DMS Fleet Advisor collects data from your on-premises database and analytic servers from one or more central locations without the need to install it on every computer. Currently, DMS Fleet Advisor supports Microsoft SQL Server, MySQL, Oracle, and PostgreSQL database servers.
- Based on data discovered from your network, DMS Fleet Advisor builds an inventory that you can review to determine which database servers and objects to monitor. As details about these servers, databases, and schemas are collected, you can analyze the feasibility of your intended database migrations.

AWS DMS COMPONENTS

- **Schema and code migration**
- DMS Schema Conversion in AWS DMS makes database migrations between different types of databases more predictable. You can use DMS Schema Conversion to evaluate the complexity of your migration for your source data provider, and then use it to convert database schemas and code objects. You can then apply the converted code to your target database.
- At a high level, DMS Schema Conversion operates with the following three components: instance profiles, data providers, and migration projects. An instance profile specifies network and security settings. A data provider stores database connection credentials. A migration project contains data providers, an instance profile, and migration rules. AWS DMS uses data providers and an instance profile to design a process that converts database schemas and code objects.

AWS DMS COMPONENTS

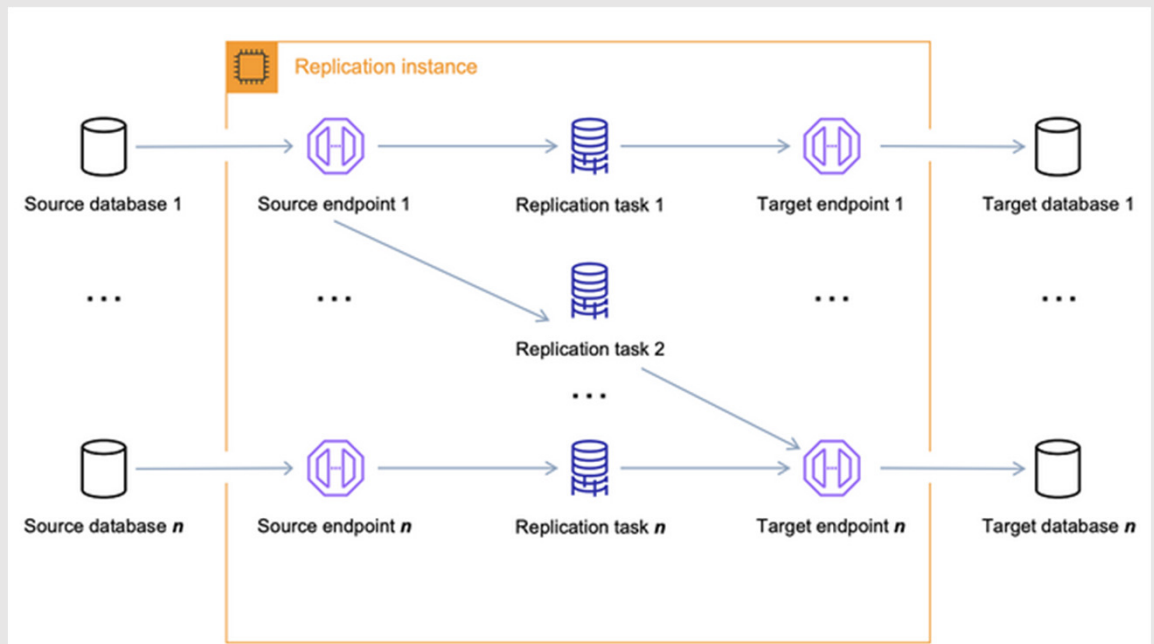
- **Replication instance**

- At a high level, an AWS DMS replication instance is simply a managed Amazon Elastic Compute Cloud (Amazon EC2) instance that hosts one or more replication tasks.
- The figure following shows an example replication instance running several associated replication tasks.

AWS DMS COMPONENTS

- **Replication instance**

- The figure following shows an example replication instance running several associated replication tasks.



AWS DMS COMPONENTS

- **Endpoint**
- AWS DMS uses an endpoint to access your source or target data store. The specific connection information is different, depending on your data store, but in general you supply the following information when you create an endpoint:
 - **Endpoint type** – Source or target.
 - **Engine type** – Type of database engine, such as Oracle or PostgreSQL.
 - **Server name** – Server name or IP address that AWS DMS can reach.
 - **Port** – Port number used for database server connections.
 - **Encryption** – Secure Socket Layer (SSL) mode, if SSL is used to encrypt the connection.
 - **Credentials** – User name and password for an account with the required access rights.

AWS DMS COMPONENTS

- **Replication tasks**

- You use an AWS DMS replication task to move a set of data from the source endpoint to the target endpoint. Creating a replication task is the last step you need to take before you start a migration.
- When you create a replication task, you specify the following task settings:
 - Replication instance – the instance to host and run the task
 - Source endpoint
 - Target endpoint
 - Migration type options, as listed following. For a full explanation of the migration type options

AWS DMS COMPONENTS

○ Replication tasks

- Migration type options, as listed following. For a full explanation of the migration type options
 - Full load (Migrate existing data) – If you can afford an outage long enough to copy your existing data, this option is a good one to choose. This option simply migrates the data from your source database to your target database, creating tables when necessary.
 - Full load + CDC (Migrate existing data and replicate ongoing changes) – This option performs a full data load while capturing changes on the source. After the full load is complete, captured changes are applied to the target. Eventually, the application of changes reaches a steady state. At this point, you can shut down your applications, let the remaining changes flow through to the target, and then restart your applications pointing at the target.

AWS DMS COMPONENTS

○ Replication tasks

- Migration type options, as listed following. For a full explanation of the migration type options
 - CDC only (Replicate data changes only) – In some situations, it might be more efficient to copy existing data using a method other than AWS DMS. For example, in a homogeneous migration, using native export and import tools might be more efficient at loading bulk data. In this situation, you can use AWS DMS to replicate changes starting when you start your bulk load to bring and keep your source and target databases in sync.

AWS DMS COMPONENTS

○ **Replication tasks**

- When you create a replication task, you specify the following task settings:
 - Target table preparation mode options, as listed following. For a full explanation of target table modes, see [Creating a task](#).
 - Do nothing – AWS DMS assumes that the target tables are precreated on the target.
 - Drop tables on target – AWS DMS drops and recreates the target tables.
 - Truncate – If you created tables on the target, AWS DMS truncates them before the migration starts. If no tables exist and you select this option, AWS DMS creates any missing tables.

AWS DMS COMPONENTS

- **Replication tasks**

- When you create a replication task, you specify the following task settings:
 - LOB mode options, as listed following. For a full explanation of LOB modes, see [Setting LOB support for source databases in an AWS DMS task](#).
 - Don't include LOB columns – LOB columns are excluded from the migration.
 - Full LOB mode – Migrate complete LOBs regardless of size. AWS DMS migrates LOBs piecewise in chunks controlled by the Max LOB Size parameter. This mode is slower than using limited LOB mode.
 - Limited LOB mode – Truncate LOBs to the value specified by the Max LOB Size parameter. This mode is faster than using full LOB mode.

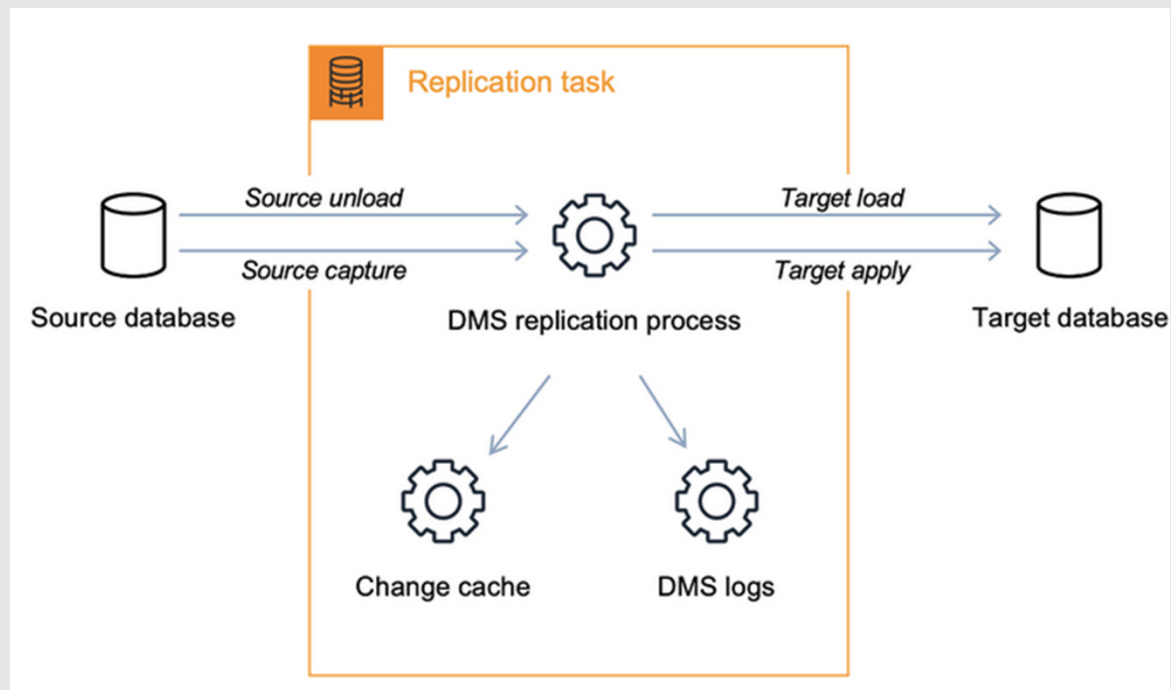
AWS DMS COMPONENTS

- **Replication tasks**

- When you create a replication task, you specify the following task settings:
 - Table mappings – indicates the tables to migrate and how they are migrated. For more information, see [Using table mapping to specify task settings](#).
 - Data transformations, as listed following. For more information on data transformations, see [Specifying table selection and transformations rules using JSON](#).
 - Changing schema, table, and column names.
 - Changing tablespace names (for Oracle target endpoints).
 - Defining primary keys and unique indexes on the target.
 - Data validation
 - Amazon CloudWatch logging

AWS DMS COMPONENTS

○ Replication tasks



AWS
SMS

AWS SERVER MIGRATION SERVICE

- AWS Server Migration Service (AWS SMS) combines data collection tools with automated server replication to speed the migration of on-premises servers to AWS.
- To use the Server Migration Connector with AWS GovCloud (US) Regions, follow these steps on your Server Migration Connector VM. The following procedure permanently converts your connector virtual appliance to an AWS GovCloud (US) connector.
 1. Install the Server Migration Connector as described in Getting Started with AWS Server Migration Service.
 2. Open the connector's virtual machine console and log in as ec2-user with the password ec2pass. Supply a new password if prompted.
 3. Run the following command: **Sudo enable-govcloud**
 4. In a web browser, access the connector VM at its IP address (<https://ip-address-of-connector/>). In the setup wizard, under AWS Region, the AWS GovCloud (US) Regions should now be the Regions listed.

AWS SERVER MIGRATION SERVICE

- **Export-controlled content**

- For AWS Services architected within the AWS GovCloud (US) Regions, the following list explains how certain components of data may leave the AWS GovCloud (US) Regions in the normal course of the service offerings. The list can be used as a guide to help meet applicable customer compliance obligations. Data not included in the following list remains within the AWS GovCloud (US) Regions.
 - Virtual machine metadata is not permitted to contain export-controlled data

AWS SERVER MIGRATION SERVICE

- Export-controlled content
 - Do not enter export-controlled data in the following fields:
 - VM names or paths
 - Virtual machine disk file paths
 - IP addresses or host names of VMs, ESXi hosts, vCenter, Hyper-V hosts, or SCVMM
 - User name of any service account or Active Directory user created for Service Migration Connector to log into vCenter, SCVMM, or Hyper-V
 - Do not enter export-controlled data into the root or boot partition of any virtual machine being imported using the AWS Server Migration Service

AWS Snowball

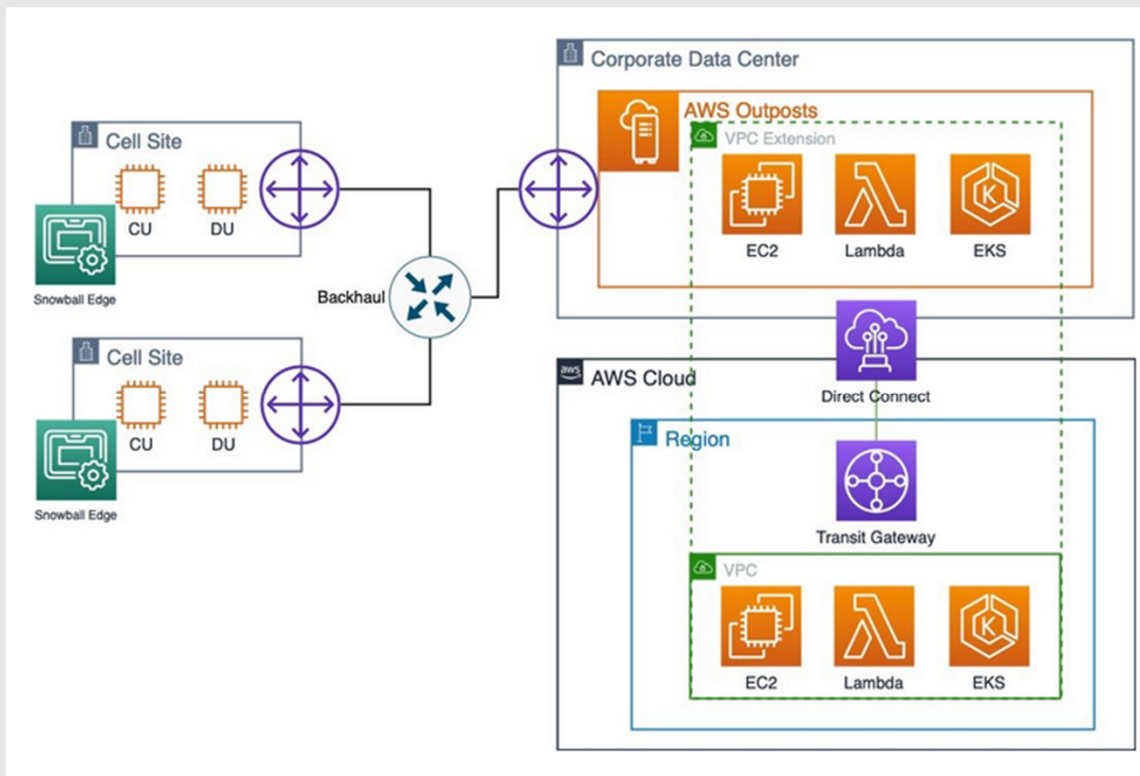
AWS SNOWBALL

- AWS Snowball uses physical storage devices to transfer large amounts of data between Amazon Simple Storage Service (Amazon S3) and your onsite data storage location at faster-than-internet speeds.
- AWS Snowball Edge is a type of Snowball device with on-board storage and compute power for select AWS capabilities.
- Snowball Edge can do local processing and edge-computing workloads in addition to transferring data between your local environment and the AWS Cloud.
- It comes pre-configured and does not have to be connected to the internet, so processing and data collection can take place within isolated operating environments.

AWS SNOWBALL

- It can operate in remote locations or harsh operating environments, such as factory floors, oil and gas rigs, mining sites, hospitals, and on moving vehicles.
- Snowball Edge allows you to run the same software at the edge and access select AWS capabilities as you would with full connectivity to AWS.
- Snowball Edge is an effective mechanism for extending cloud workloads to on-premises edge in lieu of more heavyweight options (such as AWS Outposts).
- The following diagram shows how you can make use of Snowball Edge to run OpenRAN Centralized Units (CU) or Distributed Units (DU) at Cell/Radio Sites and how this may extend to a broader deployment on AWS.

AWS SNOWBALL



AWS SNOWBALL

- Following are the key features of Snowball Edge:
 - **S3 - Compatible endpoint:** Snowball edges contain Amazon S3 and Amazon EC2 endpoints that enable the programmatic use cases.
 - **File interface:** File interface is used to read and write the data to AWS Snowball devices through a Network File System (NFS) mount point.
 - **Clustering:** You can cluster Snowball edges together to form a local storage tier and process your data on-premises to achieve 99.99% data durability across 5-10 devices and also ensures that your application continues to run even when they are not able to access the cloud.

AWS SNOWBALL

- Following are the key features of Snowball Edge:
 - **Faster Data transfer:** It transfers the data with a speed of up to 100 GB/second.
 - **Encryption:** The data transferred to Snowball Edge is automatically encrypted that are managed by Amazon Key Management Service (KMS).
 - **Run AWS Lambda function:** Each Snowball device has AWS IOT Greengrass core software that allows you to run Lambda functions.

AWS Snowball Edge

AWS SNOWBALL EDGE

- AWS Snowball Edge is a type of Snowball device with on-board storage and compute power for select AWS capabilities.
- Snowball Edge can process data locally, run edge-computing workloads, and transfer data to or from the AWS Cloud.
- Each Snowball Edge device can transport data at speeds faster than the internet.
- This transport is done by shipping the data in the devices through a regional carrier. The appliances are rugged, complete with E Ink shipping labels.
- Snowball Edge devices have three options for device configurations—Storage Optimized, Compute Optimized, and Compute Optimized with GPU.

AWS SNOWBALL EDGE

- When this guide refers to Snowball Edge devices, it's referring to all options of the device.
- When specific information applies only to one or more optional configurations of devices (such as how the Snowball Edge with GPU has an on-board GPU), it is called out specifically.

AWS SNOWBALL EDGE - FEATURES

- Large amounts of storage capacity or compute functionality for devices. This depends on the options you choose when you create your job.
- Network adapters with transfer speeds of up to 100 Gbit/second.
- Encryption is enforced, protecting your data at rest and in physical transit.
- You can import or export data between your local environments and Amazon S3, and physically transport the data with one or more devices without using the internet.
- Snowball Edge devices are their own rugged box. The built-in E Ink display changes to show your shipping label when the device is ready to ship.
- Snowball Edge devices come with an on-board LCD display that can be used to manage network connections and get service status information.

AWS SNOWBALL EDGE - FEATURES

- You can cluster Snowball Edge devices for local storage and compute jobs to achieve data durability across 3 to 16 devices and locally grow or shrink storage on demand.
- You can use Amazon EKS Anywhere on Snowball Edge devices for Kubernetes workloads.
- Snowball Edge devices have Amazon S3 and Amazon EC2 compatible endpoints available, enabling programmatic use cases.
- Snowball Edge devices support the new sbe1, sbe-c, and sbe-g instance types, which you can use to run compute instances on the device using Amazon Machine Images (AMIs).
- Snowball Edge supports these data transfer protocols for data migration:
 - NFSv3
 - NFSv4
 - NFSv4.1
 - Amazon S3 over HTTP or HTTPS (via API compatible with AWS CLI version 1.16.14 and earlier)

AWS SNOWBALL EDGE – SERVICE RELATED

- **Amazon S3 adapter** — Use for programmatic data transfer in to and out of AWS using the Amazon S3 API for Snowball Edge, which supports a subset of Amazon S3 API operations. In this role, data is transferred to the Snow device by AWS on your behalf and the device is shipped to you (for an export job), or AWS ships an empty Snow device to you and you transfer data from your on-premises sources to the device and ship it back to AWS (for an import job)"
- **Amazon S3 compatible storage on Snow Family devices** — Use to support the data needs of compute services such as Amazon EC2, Amazon EKS Anywhere on Snow, and others. This feature is available on Snowball Edge devices and provides an expanded Amazon S3 API set and features such as increased resiliency with flexible cluster setup for 3 to 16 nodes, local bucket management, and local notifications.

AWS SNOWBALL EDGE – SERVICE RELATED

- **Amazon EC2** – Run compute instances on a Snowball Edge device using the Amazon EC2 compatible endpoint, which supports a subset of the Amazon EC2 API operations. For more information about using Amazon EC2 in AWS
- **Amazon EKS Anywhere on Snow** – Create and operate Kubernetes clusters on Snow Family devices.
- **AWS Lambda powered by AWS IoT Greengrass** – Invoke Lambda functions based on Amazon S3 compatible storage on Snow Family devices storage actions made on an AWS Snowball Edge device. For more information about using Lambda.
- **Amazon Elastic Block Store (Amazon EBS)** – Provide block-level storage volumes for use with EC2-compatible instances.

AWS SNOWBALL EDGE – SERVICE RELATED

- **AWS Identity and Access Management (IAM)** – Use this service to securely control access to AWS resources.
- **AWS Security Token Service (AWS STS)** – Request temporary, limited-privilege credentials for IAM users or for users that you authenticate (federated users).
- **Amazon EC2 Systems Manager** – Use this service to view and control your infrastructure on AWS

AWS Snowmobile

AWS SNOWBALL MOBILE

- Snowmobile
 - It was announced in re: invent 2016.
 - A Snowmobile is an exabyte-scale data transfer service.
 - It can transfer large amounts of data in and out of AWS.
 - You can transfer 100 PB per Snowmobile, a 45-foot long ruggedized shipping container, pulled by a semi-trailer truck.
 - Snowmobile allows you to move massive volumes of data to the cloud, including video libraries, image repositories or even a complete migration of data center.
 - Transferring data with Snowmobile is secure, fast and cost-effective.

MODULE 8

THANK U